	local non-static	local static
time of allocation	Dynamic	Static
section of allocation	Stack	Data/BSS
frequency of allocation	per function call	once
life-time	function	Program.

```
void test_function (int i, int j)
{
    int x, y;
    static int p = 500;
    static int q;

    p = q+x;
    q = x*y;
    for (x=0; x<j; ++x)
    {
        - - -
    }
}
```

D.D.S.

D.M.S.

Data Manipulation Stmt
→ Assembly instruction
→ Machine language → text

```
class X
{
    data definition statements - non-static
    data definition statements - static
    member functions - non-static
    member functions - static

    // DO NOT GET MEMORTY
    typedef
    enum
};
```

Inside class definition, non-static data definition statements, static data definition statements, non-static member functions and static member functions are not written contiguously but they are scatterred throughout the class definition under three access specifiers viz. private, protected, public

1) static data definition statements
2) non-static member functions
3) static member functions

TIME OF ALLOCATION -> STATIC
FREQUENCY OF ALLOCATION -> ONCE
SECTION OF ALLOCATION
    1) static data definition statements -> data / BSS (based on initialization)
    2) non-static member functions, static member functions -> TEXT
LIFE - TIME == LIFETIME OF PROGRAM

```
class Test
{
    private:
        int num;
        static int s_num;   // static, data/bss, once, program,
        static void search_node(int search_data); // static, text, once, program
    protected:
        bool find(int data); // static, text, once, program
        double x, y;
    public:
        static int f1(int, int) {}    // static, text, once, program
        static int f2(float, float, float) {} // static, text, once, program
        void g1(struct Date* p) {} // static, text, once, program
        int a[5];
};
```

How to create an object from class?

lets assume that class Test is a class.

Define C-like struct, named struct Test

Scan class Test definition line by line

If the current line is non-static data definition statement

then add it to struct Test

At the end of scanning, what will you have?

All non-static data definition statements in class will be

present in the same order as they appear in class,

in struct Test. (access specifier of these non-static data

definitions statements don't matter in memory management consideration)

```
struct Test
{
    int num;
    double x;
    double y;
    int a[5];
};
```

There are two ways to allocate object:

1) data definition statement

global initialized	-> BSS
global uninitialized	-> BSS
global with const qualifier	-> RODATA
local	-> STACK
local with static qualifier	-> BSS
local with static + const qualifier	-> RODATA

2) new operator -> HEAP

(I will explain free store terminology later)

Objects which are allocated in BSS or RODATA

are allocated static and allocated only once and their life-time

is equal to life-time of entire application

Objects which are allocated on stack

are allocated dynamically, per function call and their life time

is equal to life of BLOCK in which they reside

Objects that are allocated on heap
are allocated dynamically, allocated once, and their life time
is controlled by programmer